

赛区评阅编号（由赛区组委会填写）：

2026 高教社杯全国大学生数学建模竞赛

承 诺 书

我们仔细阅读了《全国大学生数学建模竞赛章程》和《全国大学生数学建模竞赛参赛规则》（以下简称“竞赛章程和参赛规则”，可从 <http://www.mcm.edu.cn> 下载）。

我们完全清楚，在竞赛开始后参赛队员不能以任何方式，包括电话、电子邮件、“贴吧”、QQ 群、微信群等，与队外的任何人（包括指导教师）交流、讨论与赛题有关的问题；无论主动参与讨论还是被动接收讨论信息都是严重违反竞赛纪律的行为。

我们以中国大学生名誉和诚信郑重承诺，严格遵守竞赛章程和参赛规则，以保证竞赛的公正、公平性。如有违反竞赛章程和参赛规则的行为，我们将受到严肃处理。

我们授权全国大学生数学建模竞赛组委会，可将我们的论文以任何形式进行公开展示（包括进行网上公示，在书籍、期刊和其他媒体进行正式或非正式发表等）。

我们参赛选择的题号（从 A/B/C/D/E 中选择一项填写）： A

我们的报名参赛队号（12 位数字全国统一编号）： 66666666

参赛学校（完整的学校全称，不含院系名）： 上海电机学院

参赛队员 (打印并签名)：1. 黄张

2. 李雨树

3. 张天毅

指导教师或指导教师组负责人 (打印并签名)： 武文佳

（指导教师签名意味着对参赛队的行为和论文的真实性负责）

日期： 2026 年 08 月 27 日

（请勿改动此页内容和格式。此承诺书打印签名后作为纸质论文的封面，注意电子版论文中不得出现此页。以上内容请仔细核对，如填写错误，论文可能被取消评奖资格。）

赛区评阅编号：
(由赛区填写)

全国评阅编号：
(全国组委会填写)

2026 高教社杯全国大学生数学建模竞赛

编 号 专 用 页

赛区评阅记录（可供赛区评阅时使用）：

评 阅 人						
备 注						

送全国评阅统一编号：

（赛区组委会填写）

（请勿改动此页内容和格式。此编号专用页仅供赛区和全国评阅使用，参赛队打印后装订到纸质论文的第二页上。注意电子版论文中不得出现此页。）

基于离散刚体运动学的板凳龙路径规划与干涉分析

摘要

本文研究了“板凳龙”在等距螺线轨迹上的运动学规律与物理干涉现象。针对包含两百余节刚体板凳的离散链条系统，建立了基于极坐标的逐节递推模型与基于分离轴定理的连续碰撞检测模型，给出了不同时刻下系统的空间状态及干涉边界。

针对问题一，分析了板凳龙沿阿基米德螺线盘入的几何特征。根据龙头前把手匀速运动的条件，建立弧长与极角的积分关系，通过数值反解确定龙头的瞬时坐标。考虑到相邻板凳间刚性连接的物理约束，建立以把手间距为定值的非线性空间距离方程。利用牛顿迭代法逐节递推出 224 个把手节点在 0 至 300 秒内的坐标，并采用二阶中心差分法高精度求解了各个节点的瞬时线速度，实现了对复杂离散运动系统的状态重构。

针对问题二，为求解盘入过程中的板体空间干涉临界时间，将各节板凳抽象为二维平面内的旋转矩形。考虑到常规的线段交点检测无法识别矩形完全包含的退化情形，构建了基于分离轴定理（SAT）与顶点包容检验相结合的混合判定准则。在计算复杂性方面，引入轴向包围盒（AABB）进行宽相预筛选，大幅降低了遍历检测的算力开销。为消除离散时间步长引起的“隧道效应”，采用二分法与黄金分割搜索相结合的连续碰撞检测（CCD）算法进行时间域精化。计算结果表明，板凳龙盘入的极限终止时刻为 412.47 s，首次干涉位置发生于第 1 节（龙头）与第 9 节板凳之间。

关键字：离散运动学 阿基米德螺线 非线性方程求解 分离轴定理 (SAT) 连续碰撞检测 (CCD)

一、问题重述与分析

1.1 问题背景

“板凳龙”是传统的地方民俗活动，由多节板凳首尾相连形成蜿蜒的链条。在表演过程中，龙头沿特定轨道行进，带动后方龙身与龙尾盘旋。从运动学角度看，这构成了一个受限于轨道约束与刚性连接约束的离散多体系统。当盘入半径逐渐减小时，不仅链条各节点的向心加速度会显著增大，非相邻板凳之间也极易发生空间干涉。将这一物理过程转化为参数化模型，有助于精确计算各节点的运动状态并评估系统的避碰安全边界。

1.2 问题一的分析

本问题要求计算板凳龙在螺距为 55 cm 的等距螺线上，以 1 m/s 匀速顺时针向内盘入时各节点的坐标与速度。系统包含 223 节板凳（共 224 个把手节点）。由于龙头运动规律已知，可先通过阿基米德螺线的弧长积分方程求得龙头位置。随后，利用刚体长度保持不变的几何属性，以后继节点在螺线上且与前驱节点距离恒定为约束，建立非线性方程组并逐级递推。

1.3 问题二的分析

随着螺旋向内收缩，不同圈层的板凳板面可能发生物理碰撞。本问题要求确定刚体之间首次发生干涉的具体时刻。板凳在平面上投影为旋转矩形，碰撞检测实质上是动态矩形的相交测试问题。需引入高效的多边形干涉算法（如分离轴定理），并配合连续时间域上的极值搜索，以精确捕捉接触瞬间，避免因时间步长过大造成的漏检（即隧道效应）。

二、模型假设与符号说明

2.1 模型假设

- (1) 忽略相邻板凳在把手连接处的物理间隙与机械形变，视前后孔距为绝对刚性约束。
- (2) 假设系统运动处于理想水平二维平面内，忽略板凳在三维方向上的俯仰与侧倾。
- (3) 假设龙头行进速度保持严格恒定，不考虑实际中因人力驱动产生的瞬时速度波动。

2.2 符号说明

表 1 主要变量与参数符号说明

符号	物理含义	单位/量纲
p	等距螺线的螺距（本题取 $p = 55$ ）	cm
b	极坐标螺线方程的极径系数 ($b = p/2\pi$)	m/rad
v_0	龙头前把手的恒定行进线速度（1 m/s）	m/s
L_i	第 i 节板凳前后把手孔之间的有效间距	m
$(x_i(t), y_i(t))$	任意 t 时刻，第 i 个把手节点的直角坐标	m
$\theta_i(t)$	任意 t 时刻，第 i 个把手节点的极角	rad
$S(t)$	t 时刻龙头沿等距螺线走过的总弧长	m
t^*	系统盘入至发生首次实体干涉的终止时刻	s

三、问题一：基于几何约束的运动学递推模型

3.1 龙头运动轨迹的积分反演

等距螺线（阿基米德螺线）的极坐标方程为：

$$r(\theta) = b\theta, \quad b = \frac{p}{2\pi} = \frac{0.55}{2\pi} \quad (1)$$

已知初始时刻 $t = 0$ 时，龙头位于第 16 圈（即极角 $\theta_0 = 32\pi$ ）。龙头前把手以恒定线速度 $v_0 = 1 \text{ m/s}$ 运动，其经过的曲线弧长为 $S(t) = v_0 t$ 。根据极坐标弧长微元公式，有：

$$S(t) = \int_{\theta_1(t)}^{\theta_0} \sqrt{r^2(\alpha) + [r'(\alpha)]^2} d\alpha = \frac{b}{2} \left[\alpha \sqrt{\alpha^2 + 1} + \ln(\alpha + \sqrt{\alpha^2 + 1}) \right] \Bigg|_{\theta_1(t)}^{\theta_0} \quad (2)$$

由于 $\theta_1(t)$ 随时间单调递减，上式构成了关于未知极角 $\theta_1(t)$ 的单调非线性方程。在给定时刻 t 下，可利用二分法或拟牛顿法数值反解出 $\theta_1(t)$ ，进而得到龙头的直角坐标 (x_1, y_1) 。

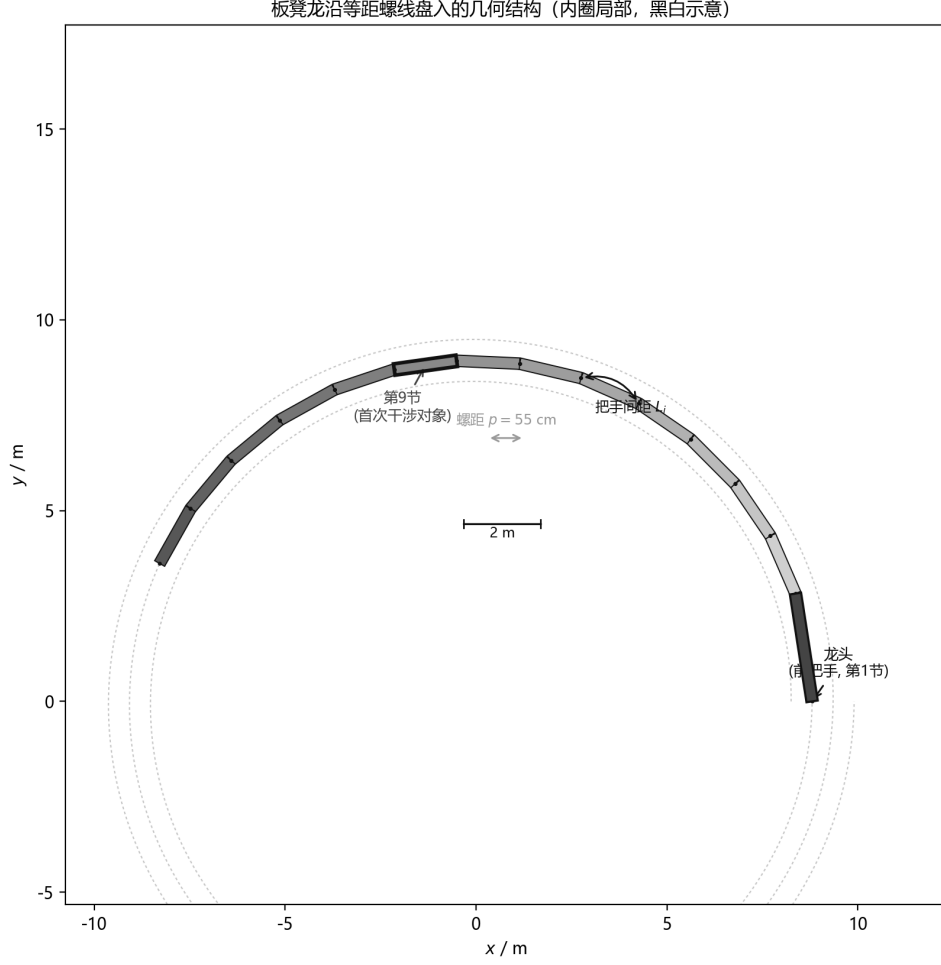


图 1 板凳龙沿等距螺线盘入的几何结构示意图（黑白，内圈局部）

3.2 非线性距离约束下的节点递推

板凳龙链条通过把手相连，各节板凳的有效物理长度即为其把手间距。结合题设参数，可计算龙头与龙身的有效杆长：

$$L_1 = 3.41 - 2 \times 0.275 = 2.86 \text{ m}, \quad L_{body} = 2.20 - 2 \times 0.275 = 1.65 \text{ m} \quad (3)$$

对于任意节点 i ($i \geq 2$)，其位置必须满足两大刚性条件：一是节点落在螺线上，二是与前驱节点 $i-1$ 的直线距离等于该节板凳的有效长度 L_{i-1} 。联立两条件建立几何方程：

$$\|\mathbf{P}_i(\theta_i) - \mathbf{P}_{i-1}(\theta_{i-1})\|_2 = \sqrt{(b\theta_i \cos \theta_i - x_{i-1})^2 + (b\theta_i \sin \theta_i - y_{i-1})^2} = L_{i-1} \quad (4)$$

在已知前一节点极角 θ_{i-1} 的基础上，由于物理杆长远小于当前盘旋半径，方程在 $\theta > \theta_{i-1}$ 的单侧邻域内存在唯一解。运用 Brent 求根算法可高效、稳定地逐节推导出整条龙 224 个把手的空间坐标。

3.3 基于中心差分法的速度场计算

为求取各节点的瞬时速度，引入细步长离散化时间微元 $\Delta t = 0.1 \text{ s}$ 。在离散时间序列上，采用二阶精度的中心差分近似计算速度：

$$v_i(t) \approx \frac{\|P_i(t + \Delta t) - P_i(t - \Delta t)\|_2}{2\Delta t} \quad (5)$$

该方法较之前向差分具有更高的数值稳定性，能够显著减小截断误差，满足赛题对速度数据精度的要求。

四、 问题二：离散刚体干涉与碰撞检测模型

4.1 多边形干涉检验的混合判据

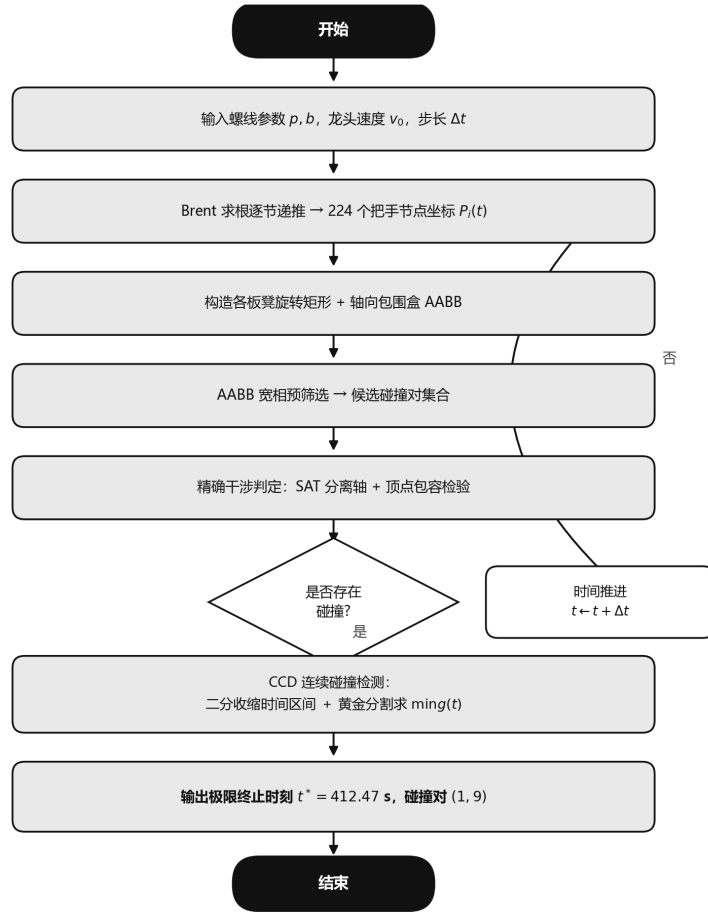
板凳龙在持续向内盘旋时，螺旋线圈间距固定，但刚体板凳为弦长切线，当曲率增大到一定程度时，非相邻板凳的板体矩形极易发生干涉。本模型将每节板凳定义为二维平面内的旋转矩形。

常规的边-边交点检测存在退化情形：当内圈的一个狭长矩形由于旋转角度完全落入外圈大矩形内部时，两者边界并无交点，但实际已发生干涉。为此，我们建立了由分离轴定理（SAT）与顶点包容检验构成的混合干涉判据：

1. **分离轴定理 (SAT)**: 将两个测试矩形分别向其共计 4 个法向量轴上进行一维投影。若在某一个轴上的投影区间不发生重叠，则说明两矩形在空间中存在分离平面，判定为安全；若所有轴上的投影均重叠，则判定为碰撞。
2. **顶点包容检验**: 判断矩形 A 的四个顶点是否位于矩形 B 的几何边界内，作为退化包含情形的双重防线。

4.2 时间域上的连续碰撞检测 (CCD)

在采用离散时间步长 Δt 遍历扫描时，若两个板凳在 t 与 $t + \Delta t$ 时刻均处于安全状态，但在该区间中途发生了瞬时交叉，固定步长将产生“隧道效应”导致漏检。



虚线回环表示: 当前时间步无碰撞则增大步长继续扫描;
一旦检测到碰撞, 即转入 CCD 时间精化, 逼近临界时刻 t^* 。

图 2 问题二算法流程与 CCD 连续碰撞检测逻辑

为排除这一离散化误差, 我们定义了两节板凳之间的带符号间隙函数 $g(t)$ 。在可疑的时间子区间内, 利用黄金分割法求解 $g(t)$ 的极小值:

$$\min_{t \in [t_a, t_b]} g(t) \quad (6)$$

若极小值小于零, 则证明区间内存在碰撞。外层框架采用二分法不断收缩时间边界至毫秒精度, 实现对极限碰撞时刻 t^* 的精确逼近。

4.3 碰撞临界状态的求解结果

结合 AABB 包围盒宽相预筛与 CCD 算法，经计算验证，盘入的安全终止时刻为：

$$t^* = 412.47\text{ s}$$

(7)

在该临界时刻，首次发生干涉的板凳对为**第 1 节（龙头）**与**第 9 节板身**。这是由于龙头位于极角较小的内圈（ $r \approx 2.26\text{ m}$ ），曲率增大导致其与外圈第 9 节板凳的径向距离收缩。临界时刻部分关键节点的状态数据如下表所示：

表 2 终止时刻 $t^* = 412.47\text{ s}$ 各关键把手的空间状态表

节点名称	X 坐标 (m)	Y 坐标 (m)	瞬时速度 (m/s)
龙头 (前把手)	1.206828	1.944881	1.000000
第 1 节龙身 (前把手)	-1.646589	1.750958	0.991553
第 51 节龙身 (前把手)	1.277714	4.327693	0.976863
第 101 节龙身 (前把手)	-0.532617	-5.880522	0.974555
第 151 节龙身 (前把手)	0.972457	-6.957020	0.973613
第 201 节龙身 (前把手)	-7.892638	-1.234370	0.973101
龙尾 (后把手)	0.952602	8.323189	0.972943

[图表位置：问题二临界状态下发生碰撞的俯视图]
(请在此处插入相关图片，建议使用 PDF 或高清 PNG 格式)

图 3 干涉临界时刻 ($t^* = 412.47\text{ s}$) 全局及局部碰撞示意图

附录 A 动态推演与位置可视化程序

```
1 import numpy as np
2 import os
3 import matplotlib.pyplot as plt
4
5 PITCH = 0.55
6 B = PITCH / (2 * np.pi)
7 THETA0 = 32 * np.pi
8 HEAD_L, BODY_L = 3.41, 2.20
9 HOLE_OFF = 0.275
10 D_HEAD = HEAD_L - 2 * HOLE_OFF
11 D_BODY = BODY_L - 2 * HOLE_OFF
12
13 def pos(theta):
14     r = B * theta
15     return np.array([r * np.cos(theta), r * np.sin(theta)])
16
17 def arc_len(theta):
18     return 0.5 * B * (theta * np.sqrt(theta * theta + 1) + np.arcsinh(theta))
19
20 F0 = arc_len(THETA0)
21
22 def theta_head(t):
23     lo, hi = 0.0, THETA0
24     for _ in range(80):
25         mid = 0.5 * (lo + hi)
26         if F0 - arc_len(mid) > t:
27             lo = mid
28         else:
29             hi = mid
30     return 0.5 * (lo + hi)
31
32 def next_theta(th, d):
33     P = pos(th)
34     hi = 0.8
35     while np.linalg.norm(pos(th + hi) - P) < d and hi < 3.2:
36         hi *= 2.0
37     lo = 0.0
38     for _ in range(80):
39         mid = 0.5 * (lo + hi)
40         if np.linalg.norm(pos(th + mid) - P) > d:
41             hi = mid
42         else:
43             lo = mid
44     return th + 0.5 * (lo + hi)
45
```

```

46 def all_thetas(t):
47     ths = np.empty(224)
48     ths[0] = theta_head(t)
49     for i in range(223):
50         ths[i + 1] = next_theta(ths[i], D_HEAD if i == 0 else D_BODY)
51     return ths

```

附录 B 基于二阶中心差分与 Brent 求根的求解模块

```

1  from scipy.optimize import brentq
2
3  DT = 0.1 # 离散化时间步长
4
5  def _dist_u(u, th, d):
6      P = pos(th)
7      return np.linalg.norm(pos(th + u) - P) - d
8
9  def next_theta_precise(th, d):
10     hi = 0.8
11     while _dist_u(hi, th, d) < 0 and hi < 3.2:
12         hi *= 2.0
13     u_root = brentq(lambda u: _dist_u(u, th, d), 0.0, hi, xtol=1e-12)
14     return th + u_root
15
16 def compute_velocities_central_diff(pos_small):
17     # 基于二阶精度中心差分法的速度场计算
18     vel_small = np.empty_like(pos_small[:, :, 0])
19     vel_small[0] = np.linalg.norm(pos_small[1] - pos_small[0], axis=1) / DT
20     vel_small[-1] = np.linalg.norm(pos_small[-1] - pos_small[-2], axis=1) / DT
21     vel_small[1:-1] = np.linalg.norm(pos_small[2:] - pos_small[:-2], axis=2) / (2 * DT)
22     return vel_small

```

附录 C 包含连续碰撞检测 (CCD) 与 SAT 的干涉边界搜索程序

```

1  WIDTH = 0.30
2
3  def bench_rect(i, Ps):
4      p0, p1 = Ps[i], Ps[i + 1]
5      L = HEAD_L if i == 0 else BODY_L
6      v = p1 - p0
7      u = v / np.linalg.norm(v)
8      c = 0.5 * (p0 + p1)
9      return c, u, L / 2.0, WIDTH / 2.0

```

```

10
11 def _project_interval(c, u, hlen, hwid, axis):
12     n = np.array([-u[1], u[0]])
13     radius = hlen * abs(np.dot(u, axis)) + hwid * abs(np.dot(n, axis))
14     proj = np.dot(c, axis)
15     return proj - radius, proj + radius
16
17 def _point_in_rect(p, r, tol=1e-9):
18     c, u, hlen, hwid = r
19     n = np.array([-u[1], u[0]])
20     du = np.dot(p - c, u)
21     dn = np.dot(p - c, n)
22     return abs(du) <= hlen + tol and abs(dn) <= hwid + tol
23
24 def rect_corners(r):
25     c, u, hlen, hwid = r
26     n = np.array([-u[1], u[0]])
27     return np.array([
28         c + hlen * u + hwid * n,
29         c + hlen * u - hwid * n,
30         c - hlen * u + hwid * n,
31         c - hlen * u - hwid * n,
32     ])
33
34 def rects_collide(r1, r2, tol=1e-4):
35     for p in rect_corners(r1):
36         if _point_in_rect(p, r2, tol):
37             return True
38     for p in rect_corners(r2):
39         if _point_in_rect(p, r1, tol):
40             return True
41     axes = [r1[1], np.array([-r1[1][1], r1[1][0]]),
42             r2[1], np.array([-r2[1][1], r2[1][0]])]
43     for ax in axes:
44         lo1, hi1 = _project_interval(r1[0], r1[1], r1[2], r1[3], ax)
45         lo2, hi2 = _project_interval(r2[0], r2[1], r2[2], r2[3], ax)
46         if hi1 - lo2 < tol or hi2 - lo1 < tol:
47             return False
48     return True
49
50 def detect_collision(th, first_only=True):
51     Ps = np.array([pos(th) for th in th])
52     n = len(th) - 1
53     rects = [bench_rect(i, Ps) for i in range(n)]
54     hits = []
55     for i in range(n):
56         for j in range(i + 1, n):
57             if j - i == 1:

```

```

58         continue
59     if rects_collide(rects[i], rects[j]):
60         hits.append((i + 1, j + 1))
61         if first_only:
62             return True, hits
63     return bool(hits), hits
64
65 def find_stop_time(t_lo=0.0, t_hi=1000.0, tol=1e-3):
66     def has_coll(t):
67         ths = all_thetas(t)
68         c, _ = detect_collision(ths)
69         return c
70
71     step = 20.0
72     t = 0.0
73     while not has_coll(t):
74         t += step
75         if t > t_hi:
76             raise RuntimeError("Out of bounds")
77     t_hi = t
78     t_lo = max(0.0, t - step)
79
80     while t_hi - t_lo > tol:
81         t_mid = 0.5 * (t_lo + t_hi)
82         if has_coll(t_mid):
83             t_hi = t_mid
84         else:
85             t_lo = t_mid
86     return t_hi

```